Deploy POC — TabGis (Linux + Docker)

Guia único para subir a stack, carregar o landbase do **Estado de São Paulo** e expor o frontend. Complementa os runbooks de dados em `docs/mds/RUNBOOK_BASEMAP_SP_ESTADO.md`.

Onde o código está versionado

O quê	Onde
Repositório Git	`https://github.com/wesleyspo/TabGis.git`
Versão actual do marco V0	**v0.1.0** (*tag* Git) + ficheiro `VERSION` na raiz
Cópia no servidor	Caminho local do projecto (ex.: `/home/wesley/projects/TabGis` no homeserver)

Detalhes de SemVer, *tags* e o que fica fora do Git (MBTiles, `data/raw/`) estão em `docs/mds/CONTEXT_STATE.md` → secção **Versionamento e código-fonte**.

---

0. Acesso remoto — outro notebook via Cursor SSH

O servidor de execução é o `homeserver` (Linux). O desenvolvimento pode ser feito a partir de **qualquer notebook** usando o Cursor com Remote SSH.

Configurar SSH no notebook cliente

Editar (ou criar) `~/.ssh/config`:

```
Host homeserver
    HostName <IP-do-homeserver>   # ex.: 192.168.1.100
    User wesley
    IdentityFile ~/.ssh/id_rsa    # chave privada local
```

Se ainda não tiver par de chaves SSH:

```
# No notebook cliente
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa
ssh-copy-id -i ~/.ssh/id_rsa.pub wesley@<IP-do-homeserver>
```

Abrir projecto no Cursor

1. `Ctrl+Shift+P` → `Remote-SSH: Connect to Host` → `homeserver`
2. Dentro da sessão remota: `File → Open Folder` → `/home/wesley/projects/TabGis`
3. O ficheiro `.cursorrules` é lido automaticamente — as regras de contexto da IA ficam activas

Aceder ao frontend

- **Mesma rede LAN:** `http://<IP-do-homeserver>:8088/`
- **SSH tunnel (fora da LAN):**

  ```
  ssh -L 8088:localhost:8088 -L 8080:localhost:8080 homeserver
  # Depois abrir: http://localhost:8088/
  ```

Verificar stack antes de trabalhar

```
docker ps --format "table {{.Names}}\t{{.Status}}"
# Esperado: todos "Up ... (healthy)" ou "Up ..."
```

Se algum container estiver ausente:

```
docker compose -f infra/docker-compose.yml up -d
```

---

1. Pré-requisitos (no homeserver)

- Docker e Docker Compose
- No host: `psql`, `shp2pgsql`, `osm2pgsql`, `curl`, `python3`

---

2. Subir serviços

```
docker compose -f infra/docker-compose.yml up -d --build
```

Na primeira vez ou após alterar `backend/`, use `--build` para construir a imagem do serviço `api`.

Serviços iniciados:

Container	Função	Porta
`db`	PostgreSQL + PostGIS	**5433** (host; mapeado para 5432 no container)
`geoserver`	WMS / WFS	8080
`tileserver`	MBTiles raster (satélite local)	8081
`api`	Node/Express — traduções (`/api/translations/:lang`)	3000
`web`	nginx — frontend + proxy `/geoserver`, `/tiles`, `/api`	8088
`mongodb`	Logs (uso futuro)	27017

> **Ordem de startup importante:** o `web` (nginx) deve ser iniciado **depois** do `tileserver`. Se recriares o `tileserver` individualmente, reinicia o `web` em seguida:
>
> ```
> docker compose -f infra/docker-compose.yml restart tileserver web
> ```

Schemas `config` / `landbase` / `operations` e traduções (API)

Se a base for nova, faltarem tabelas `config.*` / `operations.*`, ou após `git pull` que altere estes scripts, aplicar **nesta ordem** (na raiz do projecto):

```
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/01_ini
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/02_pop
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/03_ope
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/04_gis
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/05_ope
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/06_wor
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/06_wor
docker compose -f infra/docker-compose.yml exec -T db psql -U tabgis_admin -d tabgis_db < infra/sql/07_wor
```

Script	Conteúdo principal
`01_init_schemas.sql`	Schemas `landbase`, `config`, `security`, `audit`; parâmetros, traduções, idiomas; `security.users` (migração idempotente de `security.user`)
`02_populate_translations.sql`	`INSERT … ON CONFLICT DO UPDATE` das chaves de UI (EN/PT/ES/IT) consumidas pelo frontend (`data-i18n`) e pela API
`03_operations_schema.sql`	Schema `operations` (`team`, `work_order` com `owner_id` / `locked_by_id`, `team_order_displacement`, `team_route`), `audit.operations_row_archive`, triggers `BEFORE DELETE`

Script	Conteúdo principal
`04_gis_oms_mwm_domain.sql`	Schemas `ref`, `gis`, `oms`, `mwm`; `audit.domain_event`; DDL em `docs/mds/Database_Structure_Context.md`; dados mínimos
`05_operational_demo_data.sql`	Dados fictícios operacionais (utilizadores, equipas com posição, ordens) — idempotente
`06_work_order_sheet_model.sql`	Folha OMS: colunas decisórias em `work_order`, tabelas `work_order_task` / `work_order_complaint`, dados de exemplo idempotentes
`06_work_order_sheet_model_functions_triggers.sql`	Função `generate_work_order_code()` (YYYYMMDD + sequência diária America/São_Paulo), trigger de `order_code`, triggers de contagem de reclamações e arquivo pré-delete em tarefas/reclamações
`07_work_order_oms_grid_columns.sql`	Grelha OMS: `started_at`, `callback_auto` em `operations.work_order` (início operacional e callback automático)

API (`api`): após `02_populate_translations.sql`, reiniciar: `docker compose -f infra/docker-compose.yml restart api`. Após `04_gis_oms_mwm_domain.sql`, `06_*` ou alterações ao `backend/`, reconstruir: `docker compose -f infra/docker-compose.yml up -d --build api` (inclui `POST /api/work-orders/:id/tasks`, `PATCH /api/work-orders/:id/tasks/:task_id`).

SLD no GeoServer: ficheiros versionados em `infra/geoserver/styles/*.sld`. Um `git pull` que altere estilos (ex.: recorte alvo — `base_sp_bairros_alvo.sld`, `base_sp_municipios_alvo_highlight.sld`) **não** actualiza o GeoServer sozinho: na consola **Styles**, substituir o XML do estilo ou repetir o fluxo de publicação do projecto. Detalhe das camadas e notas visuais: `docs/mds/CONTEXT_STATE.md` (secção *Estilos GeoServer — recorte alvo*).

---

3. Importar landbase estadual e publicar GeoServer

```
./infra/scripts/setup_landbase_sp_estado.sh
```

O script:

- Garante extensões `postgis` e `hstore` e o schema `landbase`
- Importa shapefiles IBGE (UF + municípios), deriva `uf_sp` e `municipios_sp`
- Importa OSM: tenta `data/raw/osm/sao-paulo-latest.osm.pbf`; se inválido, usa `data/raw/osm/sudeste-latest.osm.pbf` com `--bbox` SP
- Cria `landbase.vias_sp` com `ST_Transform(way, 4674)` para alinhar com IBGE
- Cria workspace `base_sp`, datastore PostGIS e camadas WMS; aplica SLDs

Variáveis opcionais: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASS`, `GEOSERVER_URL`, `GS_USER`, `GS_PASS`, `OSM_SP`, `OSM_SUDESTE`, `SP_BBOX`

---

4. Validar GeoServer

- Interface admin: `http://homeserver:8080/geoserver/web` (user: `admin` / pass: `geoserver`)
- WMS GetCapabilities: `http://homeserver:8080/geoserver/wms?service=WMS&request=GetCapabilities`

Camadas esperadas: `base_sp:uf_sp`, `base_sp:municipios_sp`, `base_sp:vias_sp`

4.1 Recorte prioritário: municípios-alvo + bairros + vias

```
./infra/scripts/setup_municipios_alvo_sp.sh
```

Cria e publica:

- `base_sp:municipios_alvo_sp` — 24 municípios RMSP com destaque e rótulo
- `base_sp:bairros_alvo_sp` — bairros OSM dos municípios-alvo

- `base_sp:vias_alvo_sp` — vias públicas dos municípios-alvo

4.2 Satélite raster offline (MBTiles + TileServer GL)

Gerar o pacote

```
./infra/scripts/build_sp_satellite_mbtiles.sh
```

O script cria `infra/scripts/.venv/`, instala dependências e grava `data/tiles/sp_satellite.mbtiles` via ficheiro temporário (substituição atómica — o serviço não é interrompido durante a geração).

Variáveis de ambiente opcionais:

Variável	Default	Descrição
`BBOX`	`-47.2,-24.1,-46.2,-23.2`	Recorte WGS84 (capital SP)
`ZMIN`	`10`	Zoom mínimo
`ZMAX`	`16`	Zoom máximo
`DELAY_MS`	`75`	Pausa entre pedidos HTTP (ms)
`MBTILES_OUT`	`data/tiles/sp_satellite.mbtiles`	Caminho de saída

Escala por ZMAX (bbox capital SP):

ZMAX	Tiles	Tempo est.	Disco est.	Resolução
14	2 959	~4 min	~50 MB	9 m/px
15	11 331	~25 min	~200 MB	5 m/px
16	44 271	~1h15	~800 MB	2,4 m/px
17	**175 306**	**~3,5 h**	**~3 GB**	**1,2 m/px** ← default

Após geração — reiniciar serviços

```
docker compose -f infra/docker-compose.yml restart tileserver web
```

> O nginx (`web`) resolve `tileserver` na rede Docker interna apenas no arranque. O restart conjunto garante que o proxy funciona após qualquer recriação do `tileserver`.

Upgrade para ZMAX=17 (aprovado)

```
ZMAX=17 ./infra/scripts/build_sp_satellite_mbtiles.sh
# Aguardar conclusão (~3,5 h) - o ficheiro anterior permanece activo durante a geração
docker compose -f infra/docker-compose.yml restart tileserver web
```

---

5. Frontend — acesso recomendado (nginx :8088)

O serviço `web` serve `frontend/` e faz **proxy** de `/geoserver/` e `/tiles/` — o browser usa a **mesma origem** para WMS, WFS, GetFeatureInfo e tiles, sem CORS.

Abrir: **`http://homeserver:8088/`**

Configuração nginx: `infra/nginx/default.conf` | Volumes: `infra/docker-compose.yml`

5.1 Frontend alternativo — desenvolvimento local sem nginx

```
cd frontend
python3 -m http.server 5173 --bind 0.0.0.0
```

Abrir: `http://homeserver:5173` — `main.js` usa GeoServer em `http://homeserver:8080/geoserver`. **WFS e GetFeatureInfo podem falhar por CORS** — usar a secção 5 para funcionalidade completa.

Para operação totalmente offline: copiar assets Leaflet para `frontend/vendor/` e ajustar `index.html`.

---

6. Diagnóstico rápido

```
# Estado de todos os containers
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Teste tile satélite via nginx
curl -o /dev/null -w "HTTP:%{http_code}\n" \
  "http://localhost:8088/tiles/data/sp_sat/10/377/582.png"

# Metadata TileServer via nginx
curl "http://localhost:8088/tiles/data/sp_sat.json"

# Teste WMS GeoServer
curl -o /dev/null -w "HTTP:%{http_code}\n" \
  "http://localhost:8080/geoserver/base_sp/wms?SERVICE=WMS&REQUEST=GetCapabilities"
```

---

7. Firewall / LAN

Portas a libertar para acesso de outros equipamentos:

Porta	Serviço	Necessidade
8088	Frontend (recomendado)	Obrigatória
8080	GeoServer admin directo	Opcional
8081	TileServer directo	Diagnóstico
5173	Dev server estático	Opcional
5433	PostGIS (mapeamento host; interno ao Docker continua 5432)	Só admin

---

8. Referência de credenciais POC

Ver `infra/docker-compose.yml` para PostGIS e documentação da imagem GeoServer (`admin`/`geoserver` por defeito). Não usar estes valores em ambiente público sem alteração.